

BOB-168 “ acceptance (a) answered: what the runner does with a missing entry

Captured 2026-08-21. **Evidence class: SOURCE (Â§11.4.226)**. Read from `scripts/run_all_challenges.sh`, not observed at runtime “ see “œWhy not runtime” below, which is a real limitation and not a formality.

The answer is a SPLIT verdict, and both halves matter

The filed item posed two possibilities “ silent skip (advertises coverage it lacks) or hard fail (permanently red for an unrelated reason) “ and said they need different fixes. Neither is what happens.

Half 1 “ the SKIP is LOUD and counted. This half is correct (lines 88-92):

```
if [[ ! -f "${script_path}" ]]; then
    echo "  SKIP: ${script} “ not found"
    SKIP=$((SKIP + 1))
    continue
fi
```

It names the missing script, gives the reason, and increments a counter that is printed in the summary. That is Â§11.4.3 SKIP-with-reason done properly, and it rules out the “œsilent” hypothesis outright.

Half 2 “ the SKIP NEVER BLOCKS (lines 145-152):

```
echo "FAIL: ${FAIL}"
echo "SKIP: ${SKIP}"
...
if [[ "${FAIL}" -gt 0 ]]; then
    exit 1
fi
exit 0
```

The exit code reads ONLY the FAIL count. Any number of SKIPS “ including a challenge that has never existed “ exits 0.

Why that combination is the actual defect

A loud message nobody is required to act on decays into wallpaper. The roster can rot indefinitely: an entry may be listed, absent, and reported, while the suite stays green forever and no gate ever forces the question.

That is the Â§11.4.266 shape “a capability advertised in the bank with no passing challenge behind it” and it is the Â§11.4.135 principle (ABSENCE blocks exactly as a FAIL does) simply not applied at this seam.

Note the contrast with a seam in this same repo that gets it right: the release verdict-coverage rule treats a missing verdict as blocking precisely because a quiet zero and a clean result are indistinguishable to an exit code.

What this means for the item’s acceptance

Acceptance (a) is ANSWERED. Acceptance (c) “if the runner silently skips missing entries, that is its own finding” needs restating, because the premise was wrong: it does NOT skip silently. The finding is narrower and more precise: **a listed-but-absent challenge is reported and then tolerated forever**. The fix is not to make the SKIP louder; it is to decide whether an absent entry should participate in the exit code, and that is a policy choice worth stating explicitly rather than patching.

Acceptance (b) is unchanged: author the challenge or remove the entry, with Â§11.4.124 git-history discipline first, since a silently-dropped challenge is the more serious possibility.

Why not runtime evidence (Â§11.4.226 honest boundary)

Confirming this by INVOCATION would mean running the real `run_all_challenges.sh`, which is the right instrument (Â§11.4.201(11): probe the artifact through its real path, never a replica). It was not run, for two reasons that are about safety and not convenience:

1. Four subagents are live on this host and the bank includes DDoS-class and sustained-load challenges. The constitution caps test execution at 30-40% of host resources; firing that bank now would breach it and corrupt the siblings’ measurements (Â§11.4.119).
2. At a 180s timeout per entry across ~20 entries the run is up to an hour.

A hand-built replica of the loop was deliberately NOT used as a substitute: it would prove a property of the replica, not of the runner “the Â§11.4.201(11) prerequisite-vs-artifact trap.

So this reads the exit logic directly and says so. Runtime confirmation on a quiescent host remains OWED, and the claim above is scoped to source class accordingly.